Clase 8 · Sockets, familias AF_, descriptores y sockaddr_in

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · martes 22 sep 2026 · Tema 2 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. Socket y modelo cliente-servidor en software

Definición

Un **socket** es cada uno de los extremos finales de un canal de comunicación: un canal **lógico de software** entre procesos, no hardware. Los procesos pueden estar en la misma máquina o en máquinas distintas; un mismo equipo puede ser cliente y servidor a la vez.

- **Cliente:** software con rol activo que solicita un servicio.
- **Servidor:** software que espera para proveer un recurso.
- El cliente necesita **IP (o dominio) y puerto** del servidor: `dirección_IP:puerto`.

Ciclo:

1. El servidor crea su socket y se pone en **escucha** (listening).
2. El cliente crea su socket y solicita la conexión.
3. Se vinculan los dos extremos.
4. Intercambio: peticiones y respuestas.
5. **Cierre explícito** de ambos sockets; si no, el canal queda bloqueado, como un archivo que se queda abierto.

2. Comunicación síncrona vs. asíncrona

	Síncrona	Asíncrona
Orden	Lineal: envía → espera respuesta → vuelve a enviar	Sin orden estricto; varios mensajes seguidos
Ejemplos de clase	Radio/TV (propuesto por una alumna), correo postal	WhatsApp, Telegram

3. Familias de sockets (Address Family)

Macro	Uso
`AF_INET`	IPv4: 4 octetos de 0 a 255 (2^8 = 256 valores)
`AF_INET6`	IPv6

`AF_UNIX` / `AF_LOCAL`	Procesos de la **misma máquina**, asociados a un **archivo** en vez de IP:puerto. MySQL y PostgreSQL lo usan por defecto (solo localhost, por seguridad)
`AF_NETLINK`	Kernel de Linux ↔ procesos de usuario
`AF_BLUETOOTH`	Enlaces Bluetooth
`AF_PACKET`	Paquetes/tramas en capa baja (sniffers, monitoreo)

Además de la familia, cada socket tiene un **tipo**.

4. Descriptores de archivo en Unix/Linux

"**Todo es un archivo**": discos, puertos, teclado y pantalla se manejan con un **descriptor** (entero).

Descriptor	Nombre	Uso
0	stdin	Entrada (teclado)
1	stdout	Salida de éxito
2	stderr	Salida de error

Demostración en terminal:

```
ls -l /tmp > salida_exito.txt          # > toma el descriptor 1
ls -l /tmp /otro                       # el error (2) sale primero
ls -l /tmp /otro > salida.txt          # solo el éxito va al archivo
ls -l /tmp /otro 1> salida_exito.txt 2> salida_error.txt
```

La salida de error **tiene prioridad**: se imprime antes aunque `/otro` se pidió después. Separar así es lo que hacen los servidores con `server.log` y `server-error.log`.

- Pregunta en clase: ¿ambas salidas a un archivo? Sí: librerías como **Log4j**, o con `&` en la terminal.

> **Socket = descriptor 3 (examen)**
>
> Como 0, 1 y 2 están reservados, el **primer socket** de un programa en C recibe el descriptor **3**, el segundo el 4, etc.
>
> Error común: poner el mismo archivo en `1>` y `2>`; un descriptor bloquea el archivo al otro.

5. struct sockaddr_in y hostent (C, Capa 4, TCP)

TCP: confiable y orientado a conexión (UDP: datagramas no confiables).

```
struct sockaddr_in {
  sa_family_t    sin_family;  /* AF_INET */
  in_port_t      sin_port;    /* puerto, entero sin signo */
  struct in_addr sin_addr;    /* IP convertida a binario */
  unsigned char  sin_zero;    /* 8 ceros char de relleno */
};
```

- `sin_` = **socket internet**.
- `sin_addr`: la IP **no** va como texto decimal punteado; se convierte a binario.

- `sin_zero`: 8 caracteres `'0'` para igualar el tamaño de `struct sockaddr`; **opcional** en Linux moderno.
- **struct hostent:** resuelve nombres de dominio (p. ej. www.google.com): nombre oficial, alias, tipo (AF_INET), longitud (4 bytes en IPv4) y **lista de IPs** (un dominio puede tener varias, para alta disponibilidad y balanceo).

Error común: meter la IP como cadena en `sin_addr`.

Contexto y pendiente

- Demostración de pfSense (FreeBSD) por VPN en la DGP UNAM: WAN/LAN y logs de bloqueos. Hosts con máscara /11: $32 - 11 = 21 \rightarrow 2^{21}$.
- Sin tarea nueva; repasar sockaddr_in, descriptores y familias. El profesor está calificando la Tarea 2.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Concepto	Para recordar
Socket	Extremo lógico de software de un canal entre procesos
Para conectarse	IP o dominio + puerto
AF_INET / AF_UNIX	IPv4 / misma máquina vía archivo (MySQL, PostgreSQL)
0 / 1 / 2	stdin / stdout / stderr; el error sale primero
Primer socket	descriptor 3
Separar salidas	`1> exito.txt 2> error.txt`
sockaddr_in	sin_family, sin_port, sin_addr (binario), sin_zero (opcional)
hostent	Resuelve dominio → nombre, alias, tipo, longitud, lista de IPs

No olvidar

Cerrar siempre el socket. Síncrona = espera respuesta (correo postal); asíncrona = mensajes seguidos (WhatsApp).